

Q2Dynamic Link Library A scientific calculator app wants to load a math library at runtime to allow updates without recompiling. Build a shared library libmath.so with functions for absolute value and power of a complex number. Compile with gcc -fPIC -shared -o libmath.so math.c Write a test program and compile with gcc test.c -L. -lmath -Wl,-rpath,. -o test Run the program and verify output matches expected complex number magnitude

math.h

```
float abs_complex(float real, float imag);  
float power_complex(float real, float imag);
```

math.c

```
#include<math.h>  
#include "math.h"  
float abs_complex(float real, float imag) {  
    return sqrt(real*real + imag*imag);  
}  
float power_complex(float real, float imag) {  
    return pow(real*real + imag*imag, 0.5);  
}
```

test.c

```
#include<math.h>  
#include<stdio.h>  
#include "math.h"  
int main() {  
    float real, imag;  
    printf("Enter real part: ");  
    scanf("%f", &real);  
    printf("Enter imaginary part: ");  
    scanf("%f", &imag);  
    printf("Absolute value of complex number = %f\n", abs_complex(real,  
imag));  
    printf("Power of complex number = %f\n", power_complex(real,  
imag));  
    return 0;  
}
```

link

1. gcc -c -fPIC math.c -o math.o
2. gcc -shared -o libmath.so math.o
3. export LD_LIBRARY_PATH=\$PWD:\$LD_LIBRARY_PATH
4. gcc test.c -L. -lmath -lm -o mathprog
5. ./mathprog

Q3System Calls A student attendance app needs to log entries. Write a C program that creates a new process using fork(). The parent process writes "Parent attended" to a file; the child writes "Child attended" to the same file. Use fork() to create child, use getpid() and getppid() to print PIDs Parent waits for child using wait() before proceeding Both processes open the file with open() and write their entry using write()

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int fd;
    pid_t pid;
    pid = fork();
    if (pid == 0)
    {
        printf("\nChild Process");
        printf("\nChild Id = %d", getpid());
        printf("\nParent Id = %d", getppid());
        fd = open("attendance.txt", O_WRONLY | O_CREAT | O_APPEND,
0666);
        write(fd, "Child Attended\n", strlen("Child Attended\n"));
        close(fd);
    }
    else if (pid > 0)
    {
        wait(NULL);
        printf("\nParent Process");
        printf("\nParent Id = %d", getpid());
    }
}
```

```

        fd = open("attendance.txt", O_WRONLY | O_CREAT | O_APPEND,
0666);
        write(fd, "Parent Attended\n", strlen("Parent Attended\n"));
        close(fd);
    }
}

```

Q4 Multithreading An e-commerce site runs two threads: one generates order IDs and the other logs them. Both access a shared counter. Demonstrate a race condition and then fix it using a mutex lock. First run 2 threads incrementing a shared counter 5000 times without mutex and observe incorrect result. Add `pthread_mutex_lock()` and `pthread_mutex_unlock()` around the critical section. Re-run and verify final counter equals 10000.

```

#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment_without_mutex(void *arg)
{
    for (int i = 0; i < 5000; i++)
    {
        counter++;
    }
    return NULL;
}

void *increment_with_mutex(void *arg)
{
    for (int i = 0; i < 5000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()

```

```

{
    pthread_t t1, t2;
    counter = 0;
    printf("Without Mutex\n");
    pthread_create(&t1, NULL, increment_without_mutex, NULL);
    pthread_create(&t2, NULL, increment_without_mutex, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("\nFinal Counter = %d ", counter);
    counter = 0;
    printf("\nWith Mutex\n");
    pthread_mutex_init(&lock, NULL);
    pthread_create(&t1, NULL, increment_with_mutex, NULL);
    pthread_create(&t2, NULL, increment_with_mutex, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("\nFinal Counter = %d ", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}

```

Q5CPU Scheduling A call center assigns agents (processes) based on urgency. Simulate Priority (non-preemptive) scheduling for 5 calls with given burst times and priorities. Lower number = higher priority. Input: P1(bt=4,pr=2), P2(bt=3,pr=1), P3(bt=5,pr=3), P4(bt=1,pr=1), P5(bt=2,pr=4) — arrived at t=0 Sort by priority, compute waiting time and turnaround time for each process Calculate and print average waiting time and average turnaround time

```

// Q5. Priority
#include <stdio.h>
int main()
{
    int bt[5] = {4, 3, 5, 1, 2};
    int pr[5] = {2, 1, 3, 1, 4};
    int p[5] = {1, 2, 3, 4, 5};
    int wt[5], tat[5];
    int temp;
    float avg_wt = 0, avg_tat = 0;

    for (int i = 0; i < 5; i++)

```

```

{
    for (int j = i + 1; j < 5; j++)
    {
        if (pr[i] > pr[j])
        {
            temp = pr[i];
            pr[i] = pr[j];
            pr[j] = temp;

            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;

            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

wt[0] = 0;
for (int i = 1; i < 5; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

for (int i = 0; i < 5; i++)
{
    tat[i] = wt[i] + bt[i];
    avg_wt = avg_wt + wt[i];
    avg_tat = avg_tat + tat[i];
}

printf("\nProcess\tBT\tPR\tWT\tTAT\n");
for (int i = 0; i < 5; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i], bt[i], pr[i], wt[i],
tat[i]);
}

avg_wt = avg_wt / 5;
avg_tat = avg_tat / 5;
printf("\nAverage Waiting Time = %.2f ", avg_wt);
printf("\nAverage Turnaround Time = %.2f ", avg_tat);

```

```
    return 0;
}
```

Q6:Deadlock Handling A university computer lab has shared resources (projectors and USB hubs). 3 student groups hold some resources and request more. Determine if deadlock exists using the resource allocation graph approach. Represent 3 processes and 2 resource types; define Allocation and Request matrices Implement deadlock detection algorithm (check if any process can finish given available resources) Print processes involved in deadlock or "No deadlock detected"

```
// Q6. Deadlock

#include <stdio.h>

int main()
{
    int allocation[3][2] = {
        {1, 0},
        {0, 1},
        {1, 1}};

    int request[3][2] = {
        {0, 0},
        {1, 0},
        {1, 1}};

    int available[2] = {0, 0};
    int finish[3] = {0};
    int deadlock = 0;
    int found;

    while (1)
    {
        found = 0;
        for (int i = 0; i < 3; i++)
        {
            if (finish[i] == 0)
            {
                int possible = 1;
                for (int j = 0; j < 2; j++)
                {
                    if (request[i][j] > available[j])
                    {
                        possible = 0;
                    }
                }
            }
        }
    }
}
```

```

        break;
    }
}
if (possible)
{
    for (int j = 0; j < 2; j++)
    {
        available[j] = available[j] + allocation[i][j];
    }
    finish[i] = 1;
    found = 1;
    printf("P%d finished\n", i);
}
}
}
if (found == 0)
{
    break;
}
}
for (int i = 0; i < 3; i++)
{
    if (finish[i] == 0)
    {
        deadlock = 1;
    }
}

if (deadlock)
{
    printf("\nDeadlock detected in :\n");
    for (int i = 0; i < 3; i++)
    {
        if (finish[i] == 0)
        {
            printf("P%d\n", i);
        }
    }
}
else
{
    printf("\n No Deadlock Detected!");
}
}

```

```
    return 0;
}
```

Q7 Shared Memory A stock trading system requires a fast counter shared between a price-update process and a monitoring process. Use shared memory and semaphores to safely increment and read a shared stock price counter. Create shared memory segment for an integer counter using `shmget()` and `shmat()`. Writer process increments counter 10 times with a small delay. Reader process reads and prints counter value every iteration; both terminate cleanly.

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <fcntl.h>

int main()
{
    int shmidx;
    int *counter;
    sem_t *sem;
    // Create shared memory
    shmidx = shmget(IPC_PRIVATE,
                    sizeof(int),
                    IPC_CREAT | 0666);
    // Attach shared memory
    counter = (int*) shmat(shmidx,
                           NULL,
                           0);

    *counter = 0;
    // Create semaphore
    sem = sem_open("/mysem",
                   O_CREAT,
                   0666,
                   1);

    pid_t pid = fork();

    // Child Process -> Reader
    if(pid == 0)
    {
```

```

        for(int i = 0; i < 10; i++)
        {
            sem_wait(sem);

            printf("Reader reads counter = %d\n",
                *counter);

            sem_post(sem);

            usleep(200000);
        }
    }

    // Parent Process -> Writer
    else
    {
        for(int i = 0; i < 10; i++)
        {
            sem_wait(sem);

            (*counter)++;

            printf("Writer updated counter = %d\n",
                *counter);

            sem_post(sem);

            usleep(200000);
        }

        wait(NULL);

        // Cleanup
        shmdt(counter);

        shmctl(shmid,
            IPC_RMID,
            NULL);

        sem_close(sem);

        sem_unlink("/mysem");
    }
}

```

```
    return 0;
}
```

Q8IPC Classical Problems A college library has 3 reading chairs (resource). 5 students want to read. Implement the Dining Philosophers problem with 3 philosophers and 3 forks to simulate resource contention and safe resource acquisition. Create 3 philosopher threads; forks are mutexes initialized with `pthread_mutex_init()` Each philosopher thinks, picks up left fork then right fork, eats, then puts forks down Add a solution to prevent deadlock (e.g., one philosopher picks right fork first) and verify all eat at least once

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t forks[3];

void* philosopher(void* arg)
{
    int id = *(int*)arg;

    int left = id;

    int right = (id + 1) % 3;

    printf("Philosopher %d is thinking\n",
           id);

    sleep(1);

    // Deadlock prevention
    if(id == 2)
    {
        pthread_mutex_lock(&forks[right]);

        printf("Philosopher %d picked right fork\n",
               id);

        pthread_mutex_lock(&forks[left]);

        printf("Philosopher %d picked left fork\n",
               id);
    }
    else
```

```

{
    pthread_mutex_lock(&forks[left]);

    printf("Philosopher %d picked left fork\n",
           id);

    pthread_mutex_lock(&forks[right]);

    printf("Philosopher %d picked right fork\n",
           id);
}

// Eating
printf("Philosopher %d is eating\n",
       id);

sleep(1);

// Put forks down
pthread_mutex_unlock(&forks[left]);

pthread_mutex_unlock(&forks[right]);

printf("Philosopher %d finished eating\n",
       id);

return NULL;
}

int main()
{
    pthread_t p[3];

    int id[3] = {0, 1, 2};

    // Initialize mutexes
    for(int i = 0; i < 3; i++)
    {
        pthread_mutex_init(&forks[i],
                           NULL);
    }

    // Create philosophers

```

```

for(int i = 0; i < 3; i++)
{
    pthread_create(&p[i],
                  NULL,
                  philosopher,
                  &id[i]);
}

// Wait for completion
for(int i = 0; i < 3; i++)
{
    pthread_join(p[i],
                NULL);
}

// Destroy mutexes
for(int i = 0; i < 3; i++)
{
    pthread_mutex_destroy(&forks[i]);
}

return 0;
}

```

Q9Disk Scheduling An online video server stores video chunks at disk tracks: 176, 79, 34, 60, 92, 11, 41, 114. Head starts at 50, moving toward higher tracks. Find total movement using SCAN (elevator) algorithm. Implement SCAN: move head in one direction servicing requests, then reverse. Print the service order of tracks. Calculate total head movement and compare with FCFS movement for the same queue.

```

#include <stdio.h>

int main()
{
    int request[] =
    {176, 79, 34, 60, 92, 11, 41, 114};

    int higher[] =
    {60, 79, 92, 114, 176};

    int lower[] =
    {41, 34, 11};
}

```

```
int head = 50;

int total_scan = 0;

int total_fcfs = 0;

printf("SCAN Service Order:\n");

// Higher direction first
for(int i = 0; i < 5; i++)
{
    printf("%d -> %d\n",
           head,
           higher[i]);

    total_scan =
    total_scan +
    (higher[i] - head);

    head = higher[i];
}

// Reverse direction
for(int i = 0; i < 3; i++)
{
    printf("%d -> %d\n",
           head,
           lower[i]);

    total_scan =
    total_scan +
    (head - lower[i]);

    head = lower[i];
}

printf("\nTotal SCAN Movement = %d\n",
       total_scan);

// FCFS
head = 50;
```

```

printf("\nFCFS Service Order:\n");

for(int i = 0; i < 8; i++)
{
    printf("%d -> %d\n",
           head,
           request[i]);

    if(request[i] > head)
    {
        total_fcfs =
        total_fcfs +
        (request[i] - head);
    }
    else
    {
        total_fcfs =
        total_fcfs +
        (head - request[i]);
    }

    head = request[i];
}

printf("\nTotal FCFS Movement = %d\n",
       total_fcfs);

return 0;
}

```

Q10Memory Address Translation A game running in a virtual machine uses segmentation. Each segment has a base and limit. Translate virtual addresses to physical addresses and detect any out-of-bounds access. Define 3 segments: Code (base=1000, limit=400), Data (base=2000, limit=500), Stack (base=3000, limit=300) For requests (segment=0,offset=200), (segment=1,offset=600), (segment=2,offset=150) — compute physical address or print "Segmentation fault" Print result for each access with reason for fault if applicable

```

#include <stdio.h>

```

```
int main()
{
    int base[3] =
    {1000, 2000, 3000};

    int limit[3] =
    {400, 500, 300};

    int segment[] =
    {0, 1, 2};

    int offset[] =
    {200, 600, 150};

    for(int i = 0; i < 3; i++)
    {
        printf("\nRequest %d:\n",
            i + 1);

        printf("Segment = %d\n",
            segment[i]);

        printf("Offset = %d\n",
            offset[i]);

        // Check valid access
        if(offset[i] < limit[segment[i]])
        {
            int physical_address;

            physical_address =
            base[segment[i]] +
            offset[i];

            printf("Physical Address = %d\n",
                physical_address);
        }
        else
        {
            printf("Segmentation Fault!\n");

            printf("Reason: Offset exceeds limit\n");
        }
    }
}
```

```
}  
  
    return 0;  
}
```